



# Function Prototype in C

Last Updated : 29 Sep, 2023



The C function prototype is a statement that tells the compiler about the function's name, its return type, numbers and data types of its parameters. By using this information, the compiler cross-checks function parameters and their data type with function definition and function call.

Function prototype works like a function declaration where it is necessary where the function reference or call is present before the function definition but optional if the function definition is present before the function call in the program.

## Syntax

```
return_type function_name(parameter_list);
```

where,

- **return\_type**: It is the data type of the value that the function returns. It can be any data type int, float, void, etc. If the function does not return anything, void is used as the return type.
- **function\_name**: It is the identifier of the function. Use appropriate names for the functions that specify the purpose of the function.
- **parameter\_list**: It is the list of parameters that a function expects in parentheses. A parameter consists of its data type and name. If we don't want to pass any parameter, we can leave the parentheses empty.

For example,

```
int func(int, char, char *, float);
```

## Example:

Program to illustrate the Function Prototype

## C

```
// C program to illustrate the function prototype
#include <stdio.h>

// Function prototype
float calculateRectangleArea(float length, float width);

int main()
{
    float length = 5.0;
    float width = 3.0;

    // Function call
    float area = calculateRectangleArea(length, width);

    printf("The area of the rectangle is: %.2f\n", area);

    return 0;
}

// Function definition
float calculateRectangleArea(float length, float width)
{
    return length * width;
}
```

### Output

The area of the rectangle is: 15.00

## What if the function prototype is not specified?

If we ignore the function prototype, a program may compile with a warning, it may give errors and may work properly. But sometimes, it will give strange output and it is very hard to find such programming mistakes.

### Example: Program without a function prototype

The below code opens a file specified by the command-line argument, checks if the file exists, and then closes the file. The prototype of `strerror` function is not included in the code.

## C

```
// C code to check if a file exists or not.
// Prototype of strerror function is not included in the
// code.

#include <errno.h>
#include <stdio.h>

int main(int argc, char* argv[])
{
    FILE* fp;

    fp = fopen(argv[1], "r");
    if (fp == NULL) {
        fprintf(stderr, "%s\n", strerror(errno));
        return errno;
    }

    printf("file exist\n");

    fclose(fp);

    return 0;
}
```

Let us provide a filename, which does not exist in a file system, and check the output of the program on x86\_64 architecture.

## Output

```
[narendra@/media/partition/GFG]$ ./file_existence hello.c
Segmentation fault (core dumped)
```

## Explanation

The above program checks the existence of a file, provided from the command line, if a given file exists, then the program prints “file exists”, otherwise it

prints an appropriate error message.

**Why did this program crash**, instead it should show an appropriate error message. This program will work fine on x86 architecture but will crash on x86\_64 architecture. Let us see what was wrong with the code. Carefully go through the program, deliberately I haven't included the [prototype](#) of the "strerror()" function. This function returns a "pointer to the character", which will print an error message which depends on the [errno](#) passed to this function.

Note that x86 architecture is an ILP-32 model, which means integers, pointers, and long are 32-bit wide, that's why the program will work correctly on this architecture. But x86\_64 is the LP-64 model, which means long and [pointers](#) are 64-bit wide. *In C language, when we don't provide a prototype of a function, the compiler assumes that the function returns an integer.* In our example, we haven't included the "[string.h](#)" header file (strerror's prototype is declared in this file), that's why the compiler assumed that the function returns an integer. But its return type is a pointer to a character.

In x86\_64, pointers are 64-bit wide and integers are 32-bit wide, that's why while returning from a function, the returned address gets truncated (i.e. 32-bit wide address, which is the size of integer on x86\_64) which is invalid and when we try to dereference this address, the result is a [segmentation fault](#).

**Now include the "string.h" header file and check the output, the program will work correctly and gives the following output.**

```
[narendra@/media/partition/GFG]$ ./file_existence hello.c
No such file or directory
```

### Consider one more example

This code demonstrates the process of dynamic memory allocation using the [malloc function](#), but the prototype of the malloc function is not included.

---

C

```
// This code dynamically allocates memory for an integer
// using the malloc function but prototype of malloc
// function is not included in the code
```

```
#include <stdio.h>

int main(void)
{
    int* p = malloc(sizeof(int));

    if (p == NULL) {
        perror("malloc()");
        return -1;
    }

    *p = 10;
    free(p);

    return 0;
}
```

## Explanation

The above code will work fine on the IA-32 model but will fail on the IA-64 model. The reason for the failure of this code is we haven't included a prototype of the malloc() function and the returned value is truncated in the IA-64 model.

## FAQs on Function Prototype

### Q1. Difference between Function Declaration and Function Prototype

Answer:

*Following are some differences between Function and Function Prototype:*

| <i>Function Declaration</i>                                              | <i>Function Prototype</i>                                                                           |
|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <i>Function Declaration is used to tell the existence of a function.</i> | <i>The function prototype tells the compiler about the existence and signature of the function.</i> |

| <i><b>Function Declaration</b></i>                                                   | <i><b>Function Prototype</b></i>                                                                                                                              |
|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>A function declaration is valid even with only function name and return type.</i> | <i>A function prototype is a function declaration that provides the function's name, return type, and parameter list without including the function body.</i> |
| <i>Typically used in header files to declare functions.</i>                          | <i>Used to declare functions before their actual definitions.</i>                                                                                             |
| <p><b>Syntax:</b></p> <pre>return_type function_name();</pre>                        | <p><b>Syntax:</b></p> <pre>return_type<br/>function_name(parameter_list);</pre>                                                                               |

This article is compiled by **Narendra Kangralkar**.

Summer-time is here and so is the time to skill-up! More than 5,000 learners have now completed their journey from **basics of DSA to advanced level development programs** such as Full-Stack, Backend Development, Data Science.

And why go anywhere else when our [DSA to Development: Coding Guide](#) will help you master all this in a few months! Apply now to our [DSA to Development Program](#) and our counsellors will connect with you for further guidance & support.

**Previous Article**

Parameter Passing Techniques in C

**Next Article**

How can I return multiple values from a function?

## Similar Reads

### What is the purpose of a function prototype?

The Function prototype is necessary to serve the following purposes: Function prototype tells the return type of the data that the function will return. Function...

2 min read

### C Function Arguments and Function Return Values

Prerequisite: Functions in C A function in C can be called either with arguments or without arguments. These functions may or may not return values to the calling...

6 min read

### Write a one line C function to round floating point numbers

Algorithm: roundNo(num) 1. If num is positive then add 0.5. 2. Else subtract 0.5. 3. Type cast the result to int and return. Example: num = 1.67, (int) num + 0.5 =...

1 min read

### Does C support function overloading?

First of all, what is function overloading? Function overloading is a feature of a programming language that allows one to have many functions with same nam...

2 min read

### How can I return multiple values from a function?

We all know that a function in C can return only one value. So how do we achieve the purpose of returning multiple values. Well, first take a look at the declaratio...

2 min read

[View More Articles](#)

Article Tags :

[C Language](#)

[C-Functions](#)



A-143, 9th Floor, Sovereign Corporate  
Tower, Sector-136, Noida, Uttar Pradesh -  
201305



## Company

About Us  
Legal  
Careers  
In Media  
Contact Us  
Advertise with us  
GFG Corporate Solution  
Placement Training Program

## Languages

Python  
Java  
C++  
PHP  
GoLang  
SQL  
R Language  
Android Tutorial

## Data Science & ML

Data Science With Python  
Data Science For Beginner  
Machine Learning Tutorial  
ML Maths  
Data Visualisation Tutorial  
Pandas Tutorial  
NumPy Tutorial  
NLP Tutorial  
Deep Learning Tutorial

## Python Tutorial

Python Programming Examples  
Django Tutorial

## Explore

Job-A-Thon Hiring Challenge  
Hack-A-Thon  
GfG Weekly Contest  
Offline Classes (Delhi/NCR)  
DSA in JAVA/C++  
Master System Design  
Master CP  
GeeksforGeeks Videos  
Geeks Community

## DSA

Data Structures  
Algorithms  
DSA for Beginners  
Basic DSA Problems  
DSA Roadmap  
DSA Interview Questions  
Competitive Programming

## Web Technologies

HTML  
CSS  
JavaScript  
TypeScript  
ReactJS  
NextJS  
NodeJs  
Bootstrap  
Tailwind CSS

## Computer Science

GATE CS Notes  
Operating Systems



Python Projects  
Python Tkinter  
Web Scraping  
OpenCV Tutorial  
Python Interview Question

### DevOps

Git  
AWS  
Docker  
Kubernetes  
Azure  
GCP  
DevOps Roadmap

### School Subjects

Mathematics  
Physics  
Chemistry  
Biology  
Social Science  
English Grammar

### Databases

SQL  
MYSQL  
PostgreSQL  
PL/SQL  
MongoDB

### Competitive Exams

JEE Advanced  
UGC NET  
UPSC  
SSC CGL  
SBI PO  
SBI Clerk  
IBPS PO  
IBPS Clerk

### Free Online Tools

Typing Test  
Image Editor  
Code Formatters  
Code Converters

Computer Network  
Database Management System  
Software Engineering  
Digital Logic Design  
Engineering Maths

### System Design

High Level Design  
Low Level Design  
UML Diagrams  
Interview Guide  
Design Patterns  
OOAD  
System Design Bootcamp  
Interview Questions

### Commerce

Accountancy  
Business Studies  
Economics  
Management  
HR Management  
Finance  
Income Tax

### Preparation Corner

Company-Wise Recruitment Process  
Resume Templates  
Aptitude Preparation  
Puzzles  
Company-Wise Preparation  
Companies  
Colleges

### More Tutorials

Software Development  
Software Testing  
Product Management  
Project Management  
Linux  
Excel  
All Cheat Sheets

### Write & Earn

Write an Article  
Improve an Article  
Pick Topics to Write  
Share your Experiences

Currency Converter

Internships

Random Number Generator

Random Password Generator

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved